

CSE4121 — Object Oriented Design and Design Patterns

Prediction Questions WITH Complete Answers

Companion to the road-map PDF (CSE4121_1_..._v2.pdf). Every Model-Set-3 prediction question is answered here in exam style, followed by the extra colour-marked questions that Set 3 could not fit. Answers follow the full-mark recipe: name + intent, roles, UML description, Java skeleton, scenario-tied advantages.

PART 1 — Answers to Model Set 3 (the predicted paper)

Q1(a). What is design; conceptual vs technical design

Source: Slide 1 suggestion; Set 3 Q1(a)

"Design begins before a single line of code." What is design? Differentiate conceptual and technical design, naming the tool of each stage.

Design is the planning of a system's structure, behaviour and interactions before coding — the bridge from analysis (what) to implementation (how). Without it, code grows ad hoc: duplicated logic, hidden dependencies and expensive late changes.

Conceptual design

- Purpose: communicate the design to clients; understand the problem.
- Tool: CRC cards — classes, responsibilities, collaborations; no code.
- Level: objects and responsibilities; prototype / high-level simulation.

Technical design

- Purpose: give developers detailed guidance for coding.
- Tool: UML class diagrams — classes, attributes, methods, visibility, relationships.
- Level: close to code; converts directly into Java.

DUMS waste module: conceptually we agree "Citizen submits complaint, WasteManager assigns truck" (CRC); technically we draw Citizen, Complaint, WasteTruck classes with methods and associations before writing Java. Hence good design begins before code.

Q1(b). CRC cards for the DUMS complaint scenario

Source: Slide 1 suggestion (responsibility + collaboration); Set 3 Q1(b)

Model the DUMS complaint scenario with CRC cards showing responsibilities and collaborations.

A CRC card states a class name, its responsibilities (what it knows/does) and its collaborations (classes it needs).

Citizen (entity)

- R: store name/NID/address; submit complaint; track own complaint status.
- C: ComplaintPortal (boundary), Complaint.

Complaint (entity)

- R: hold description, category, status, ticket number.
- C: Citizen (owner), ComplaintManager.

ComplaintManager (control)

- R: validate complaint; assign to collection scheduler; update status.
- C: Complaint, CollectionScheduler, NotificationService.

CollectionScheduler (control)

- R: check truck availability; assign truck; queue if none.
- C: WasteTruck, Complaint.

NotificationService (control) / ComplaintPortal (boundary)

- NotificationService R: send SMS/push status updates. C: Citizen.
- ComplaintPortal R: present form, receive submission. C: Citizen, ComplaintManager.

The collaboration columns become associations in the UML class diagram — CRC is the conceptual tool, UML the technical one.

Q1(c). Entity / control / boundary classification in AAS

Source: Slide 1 suggestion (object types); Set 3 Q1(c)

Classify Field, SoilSensorReading, IrrigationScheduler, FarmerMobileScreen as entity, control or boundary with a reason.

- Field — ENTITY: a persistent real-world thing holding data (crop, area, moisture threshold).
- SoilSensorReading — ENTITY: a data object recording sensor value + time (information the system stores).
- IrrigationScheduler — CONTROL: coordinates logic — compares readings with thresholds, checks forecast, opens valves; it orchestrates other objects.
- FarmerMobileScreen — BOUNDARY: the interface between the system and the actor (farmer); it displays information and receives input.

Rule of thumb: nouns that store data → entity; verbs that coordinate → control; interaction points with actors or external systems → boundary.

Q2(a). Evaluate coupling and cohesion of the given code

Source: Slide 3 suggestion (code GIVEN — analyse, no code writing); Set 3 Q2(a)

Given a DUMS CitizenService class that mixes login, payment, complaint and notification duties and imports LoginService, PaymentGateway, ComplaintDB, SmsGateway, EmailGateway — evaluate coupling and cohesion.

```
public class CitizenService {
    private LoginService login;    private PaymentGateway pay;
    private ComplaintDB db;        private SmsGateway sms;
    private EmailGateway mail;
    public boolean login(String nid, String pass) { ... }
    public void payBill(String billId, double tk) { ... }
    public void addComplaint(Complaint c) { ... }
    public void notifyCitizen(String msg) { ... }
}
```

Coupling — HIGH (bad)

- The class depends on FIVE concrete external classes (five fields + imports); any change in a gateway or DB API forces changes here.
- Direct use of concrete types, not interfaces — violates Dependency Inversion.

Cohesion — LOW (bad)

- Four unrelated responsibilities in one class: authentication, billing, complaint storage, notification — no single purpose; violates Single Responsibility Principle.
- Many reasons to change = the Large Class smell.

Verdict and fix

The design is poor: high coupling + low cohesion. Split into AuthService, BillingService, ComplaintService, NotificationService; let them depend on interfaces; a controller coordinates them. (Per the teacher's note: analysing the given code like this is enough — no new code is required.)

Q2(b). Separation of Concerns + conceptual integrity

Source: Slide 3 marks; Set 3 Q2(b)

Explain Separation of Concerns with the SmartPhone example and define conceptual integrity.

SoC with SmartPhone

- Initial design: one SmartPhone class containing camera + phone functions → low cohesion, poor modularity; camera and call logic cannot be reused or replaced separately.
- Improved design: two concerns identified — Phone functionality and Camera functionality; separate cohesive classes Phone and Camera; SmartPhone coordinates them, providing access without internal details.

Result: each concern changes, tests and reuses independently (upgrade camera without touching call logic).

Conceptual integrity

The whole system follows one consistent philosophy — uniform naming, one error-handling strategy, one layering rule — so it reads as if designed by a single mind. Ad-hoc additions that break the convention are rejected. With SoC it keeps the architecture understandable and maintainable.

Q2(c). DRY and Principle of Least Knowledge

Source: Slide 6 marks; Set 3 Q2(c)

State the DRY principle and the Principle of Least Knowledge.

- DRY (Don't Repeat Yourself): every piece of knowledge must have a single, unambiguous implementation. Duplicated validation in five classes breaks DRY; one shared Validator gives one fix point → easier maintenance.
- Principle of Least Knowledge (Law of Demeter): a class should talk only to its direct friends. Avoid chains like `c.getCitizen().getAddress().getZone().getOfficer()`; instead call `c.escalateToOfficer()`. Fewer dependencies → easier modification.

Q3(a). 150-line method: smells and refactoring

Source: Slide 6 colour marks; Set 3 Q3(a)

A method contains 150 lines performing validation, database operations, calculations and report generation. Identify the smells, their effect on maintainability, and a refactoring approach.

Smells

- Long Method: four unrelated phases in one method.
- Low cohesion / Large Class if the owner holds several such methods.
- Violates SRP; logic cannot be reused or tested separately (DRY suffers when copies appear).

Effect on maintainability

- Hard to read and review; a bug can hide anywhere in 150 lines.
- Untestable as a unit; any change risks breaking an unrelated phase.
- New report types or validations force editing the same giant method.

Refactoring approach

- Extract Method: `validate()`, `saveToDatabase()`, `calculate()`, `generateReport()`; the original becomes a short readable sequence.
- Intention-revealing names; each method one responsibility.
- Move methods to dedicated classes where the responsibility belongs (Extract Class).
- Each piece becomes independently testable and reusable.

Q3(b). Duplicated switch in five methods

Source: Slide 6 colour marks; Set 3 Q3(b)

The same switch statement for customer types appears in five different methods. Identify the smell, the maintenance problems and a refactoring.

Smells

- Duplicated Code: one logic, five copies.
- Large Switch Statements: repeated conditionals signal a missing abstraction.

Maintenance problems

- New customer type → edit five places; miss one and behaviour is inconsistent.
- Bug fixes multiply; copies drift apart (DRY violation).
- Every copy must be modified to extend — violates Open/Closed Principle.

Refactoring

- Quick: Extract Method — one shared method used by all five call sites.
- Proper: Replace Conditional with Polymorphism — CustomerType hierarchy (Regular/VIP/Corporate) each implementing its own behaviour; the switch disappears; adding a type = adding a subclass, no old code edited (OCP).

Q3(c). Are comments always beneficial?

Source: Slide 6 colour mark; Set 3 Q3(c)

Evaluate: "Comments are always beneficial in software development."

False — not always.

- Beneficial: explaining WHY (constraints, trade-offs, legal rules), documenting APIs.
- Harmful: line-by-line WHAT comments duplicate the code, rot when it changes, and mislead.

Excessive comments often compensate for bad design: a long comment above a 40-line block means the block should be an Extract Method with a good name; many such comments in one class indicate a Large Class smell. Evaluation: prefer self-documenting code (small methods, intention-revealing names); comment only intent the code cannot express.

Q4(a). Design patterns, GoF, three categories, seven Module-1 patterns

Source: Slide 4 marks; Set 3 Q4(a)

What is a design pattern? Name the three GoF categories and list the seven Module-1 patterns with one-line intents.

A design pattern is a practical, proven, reusable solution to a recurring design problem — a template to adapt, not a library to copy. The Gang of Four (Gamma, Helm, Johnson, Vlissides) catalogued 23 classic patterns.

- Creational — controlled object creation: Singleton (exactly one instance, global access); Factory Method (creation delegated to subclasses).
- Structural — composing objects into larger structures: Adapter (bridge incompatible interfaces); Facade (simple interface over a complex subsystem); Composite (tree hierarchies, uniform treatment); Proxy (access control); Decorator (dynamic responsibilities).
- Behavioural — object communication: Observer, Command, Chain of Responsibility, State, Template Method.

Module 1's seven: Singleton, Factory Method, Adapter, Facade, Composite, Proxy, Decorator.

Q4(b). Singleton for a shared configuration object

Source: Slide 4 marks; Set 3 Q4(b)

An application needs exactly one shared configuration object. Apply Singleton with Java; when is Singleton a bad choice?

```
public class AppConfig {
    private static AppConfig instance;
    private String dbName;
    private AppConfig() {} // blocks external new
    public static AppConfig getInstance() {
        if (instance == null) instance = new AppConfig();
        return instance; // same object forever
    }
}
```

- Every module calls AppConfig.getInstance() → one shared object, lazy creation.
- Thread-safe variant: synchronised getInstance() or eager static final instance.

Bad choice when: many instances would be fine (Singleton for every shared object = hidden global state), or testability matters (global state makes unit tests order-dependent). Use only when exactly-one is a true requirement.

Q4(c). Factory Method for university courses

Source: Slide 4 marks; Set 3 Q4(c)

Design a Factory Method system creating TheoryCourse, LabCourse and ProjectCourse; describe classes and relationships.

```
abstract class Course { abstract String type(); }
class TheoryCourse extends Course { String type() { return "Theory"; } }
class LabCourse extends Course { String type() { return "Lab"; } }
class ProjectCourse extends Course { String type() { return "Project"; } }

abstract class CourseFactory { abstract Course createCourse(); } // factory method
class TheoryFactory extends CourseFactory { Course createCourse() { return new TheoryCourse(); } }
class LabFactory extends CourseFactory { Course createCourse() { return new LabCourse(); } }
class ProjectFactory extends CourseFactory { Course createCourse() { return new ProjectCourse(); } }
```

Relationships: parallel Creator and Product hierarchies; the client depends only on CourseFactory/Course — object creation is separated from usage, no new in client code, and a new course type needs only a new subclass pair (OCP).

Q5(a). MVC components and separation of concerns

Source: Slide 6 underlined + marked; Set 3 Q5(a)

What are the three components of MVC? How does MVC achieve separation of concerns in a maintainable web application?

Components

- Model: data + business rules (complaints, validation, status); knows nothing about screens.
- View: presentation — renders model data as HTML/JSON; no business rules.
- Controller: receives HTTP requests, calls the model, selects the view.

Request flow

(1) citizen submits form → (2) ComplaintController validates and calls ComplaintModel.add() → (3) controller chooses ComplaintView → (4) view reads model data → (5) HTML returned.

Why it is maintainable (SoC)

- Each concern isolated: rules (model), presentation (view), flow (controller).
- UI redesign touches only views; a new rule touches only the model; one model can serve web + mobile views.
- Each part is independently testable → maintainable and extensible.

Q5(b). MVC for a student registration system

Source: Slide 6 marks; Set 3 Q5(b)

Apply MVC to a university student registration system; identify Model, View, Controller and their communication.

- Model: Student, Course, Registration — data plus rules (seat availability, prerequisites); methods register(), drop().
- View: RegistrationFormView, CourseListView, ResultView — render HTML from model data.
- Controller: RegistrationController — receives the submission, invokes Registration.register(student, course), chooses the result view.

Communication

Browser → Controller (request); Controller → Model (business operation); Controller → View (selection); View → Model (read data). The Model never calls the View. Change localisation: new page style = new View; new credit rule = Model only.

Q6(a). Adapter for legacy payment and IoT sensor

Source: Slide 4 marks + CT-2; Set 3 Q6(a)

Legacy payment offers makePayment(); the app expects pay(). A new IoT soil sensor cannot talk to the existing irrigation controller. One pattern for both? UML + Java.

Adapter — converts an existing interface into the one the client expects, without modifying either side. Both scenarios are “right functionality, wrong interface”.

```
interface PaymentService { void pay(double amount); } // target
class LegacyPayment { public void makePayment(double a) { ... } } // adaptee
class PaymentAdapter implements PaymentService {
    private LegacyPayment legacy;
    PaymentAdapter(LegacyPayment l) { this.legacy = l; }
    public void pay(double a) { legacy.makePayment(a); } // translate
}
// client: PaymentService ps = new PaymentAdapter(new LegacyPayment());
```

UML (object adapter)

- Client → Target interface (pay / readMoisture).
- Adapter implements Target and holds the Adaptee (composition), translating calls.
- AAS: SoilSensorAdapter implements StandardSensor, wraps IoTSoilSensor; the IrrigationController is reused unchanged.

Advantages: reuse of existing tested classes; open/closed — new sensors/adapters added without editing the controller.

Q6(b). Decorator for coffee with milk, sugar, cream

Source: Slide 4 marks; Set 3 Q6(b)

A coffee application must add milk, sugar and whipped cream dynamically. Apply a suitable pattern; why does subclassing every combination fail?

```
interface Beverage { String desc(); double cost(); }
class BasicCoffee implements Beverage { ... cost = 50 ... }
abstract class CondimentDecorator implements Beverage {
    protected Beverage inner; // wraps component
}
class Milk extends CondimentDecorator { double cost() { return inner.cost() + 10; } }
class Sugar extends CondimentDecorator { double cost() { return inner.cost() + 5; } }
class Cream extends CondimentDecorator { double cost() { return inner.cost() + 15; } }
// Beverage b = new Cream(new Sugar(new Milk(new BasicCoffee())));
```

- Subclassing every combination explodes (CoffeeWithMilk, CoffeeWithMilkAndSugar, ... = 2^n classes) — the class-explosion smell; every new condiment multiplies subclasses.
- Decorators combine at RUNTIME in any order, add/remove responsibilities dynamically, one responsibility per class (SRP, OCP).

Same shape in DUMS: Notification decorated by EmailDecorator, SMSDecorator, PushDecorator.

PART 2 — Extra colour-marked questions with answers

X1. Observer: DUMS complaint status (CT-2 Q1)

Source: Behavioural suggestion + CT-2

The complaint system must automatically notify the citizen, the department officer and the city administrator on every status change. Implement.

Observer — one-to-many dependency; when the Subject changes, all registered Observers are notified automatically.

```
interface Observer { void update(String status); }
class Citizen implements Observer { ... }
class DepartmentOfficer implements Observer { ... }
class CityAdministrator implements Observer { ... }
class Complaint { // subject
    private String status;
    private List<Observer> observers = new ArrayList<>();
    void attach(Observer o) { observers.add(o); }
    void setStatus(String s) { status = s;
        for (Observer o : observers) o.update(s); }
}
```

Loose coupling: a new user type (Mayor) = a new Observer class; Complaint is untouched (OCP).

X2. Chain of Responsibility: university + bank

Source: Behavioural suggestion + CT-2 Q2

Apply CoR to Teacher → Chairman → Dean → Vice Chancellor; explain the bank cheque example.

CoR passes a request along a chain of handlers until one handles it — sender and receiver decoupled.

```
abstract class Approver {
    protected Approver next;
    void setNext(Approver n) { next = n; }
    abstract void handle(Request r);
}
class Teacher extends Approver {
    void handle(Request r) { if (r.level <= 1) approve(r); else next.handle(r); }
}
// Chairman (2), Dean (3), ViceChancellor (4): same shape
```

- Bank cheque: < Tk 10,000 → Cashier; 10,000–10,00,000 → Cashier + Senior Officer; > 10,00,000 → Senior Officer + Manager.
- Each handler checks authority and forwards otherwise; inserting a level never touches the client.

X3. Command and State (+ identification drill)

Source: Module-2 sample bank

(a) Remote control with LightOn/LightOff/FanOn using Command. (b) How State improves a giant switch on order status.

(a) Command

```
interface Command { void execute(); }
class Light { void on(){} void off(){} } // receiver
class LightOnCommand implements Command {
    Light l; LightOnCommand(Light l){ this.l = l; }
    public void execute(){ l.on(); }
}
class Remote { // invoker
    Command slot;
    void setCommand(Command c){ slot = c; }
    void press(){ slot.execute(); }
}
```

Buttons are decoupled from devices; supports undo, queueing, logging.

(b) State

Replace switch(orderStatus) with state classes Pending, Paid, Shipped, Delivered implementing OrderState; the Order context delegates to its current state. New state = new class; no switch edited (OCP).

Identification drill

- One shared object → Singleton; creation via subclass → Factory Method; incompatible interfaces → Adapter; simplify subsystem → Facade; tree hierarchy → Composite; access control → Proxy; dynamic features → Decorator; notify subscribers → Observer; approval levels → CoR; request as object/undo → Command; behaviour by state → State;

algorithm skeleton → Template Method.

X4. Composite for File/Folder

Source: Slide 4 mark

Why is Composite appropriate for a file system of File and Folder objects?

- interface FileSystemComponent { String getName(); long getSize(); }
- Leaf = File (own size); Composite = Folder (children list; getSize() sums children recursively).

The hierarchy is a tree; clients treat a single file and a whole folder uniformly (same getSize()/print() call), recursion is natural, and new node types need no client change (OCP). Same idea: City → Zone → Ward → Bin in DUMS.

X5. Facade vs Adapter vs Proxy

Source: Slide 4 marks

Differentiate Facade, Adapter and Proxy; when is each the right choice?

- Adapter: converts one interface to another so incompatible classes collaborate — functionality right, interface wrong (makePayment → pay).
- Facade: a new simple interface over a complex subsystem (HomeTheaterFacade.startMovie() hides amplifier + projector + lights) — reduces client complexity.
- Proxy: same interface as the real subject but controls access — protection, remote access, lazy loading (load a huge image only when displayed).

Incompatible existing interfaces → Adapter. Too-complex subsystem → Facade. Access control / expensive object → Proxy.

X6. Smell definitions quick sheet

Source: Slide 6 underlined

One-line definitions to memorise: anti-pattern, code smell, Large Class, Long Method, Duplicated Code, Primitive Obsession, Inappropriate Intimacy, Message Chains, Speculative Generality.

- Anti-pattern: commonly used but ineffective design practice (God Class, Golden Hammer).
- Code smell: surface symptom indicating a deeper design problem.
- Large Class: many responsibilities in one class → low cohesion.
- Long Method: one method, many unrelated steps.
- Duplicated Code: same logic copied; fixes must repeat.
- Primitive Obsession: raw types instead of domain objects (string coordinates instead of Location).
- Inappropriate Intimacy: one class digging into another's internals.
- Message Chains: a.getB().getC().do() — over-long delegation (Law of Demeter violation).
- Speculative Generality: unused hooks for imagined futures.